

Formal Mathematical Formulation of Sudoku Puzzle Optimization Problems

1 Problem B: High-Level Blueprint

Problem B assumes a fixed clue pattern $m \in \{0, 1\}^{81}$ and seeks the full Sudoku solution grid G that maximizes the difficulty of the induced puzzle $P(m, G)$ while maintaining unique solvability. Formally:

$$\max_G \text{Diff}(P(m, G)) \quad \text{s.t.} \quad |S(P(m, G))| = 1.$$

The entire workflow can be organized into five top-level modules:

1.1 Module 1: Input Space Construction

- Construct or sample a large set of complete Sudoku solution grids:

$$G_1, G_2, \dots, G_N.$$

- Generate full solutions via:
 - depth-first search completion,
 - digit permutations,
 - row/column/band transformations,
 - randomization techniques.
- The goal is to obtain a diverse and sufficiently large candidate space.

1.2 Module 2: Pattern Application

- Given the fixed pattern m , construct the puzzle induced by each full grid:

$$P(m, G)[i] = \begin{cases} G[i], & m_i = 1, \\ 0, & m_i = 0. \end{cases}$$

- This transforms each G_k into a corresponding puzzle instance requiring evaluation.

1.3 Module 3: Puzzle Feasibility Filtering

- Evaluate solvability and uniqueness:

$$|S(P(m, G))| = 1.$$

- Employ a backtracking or constraint-based solver with early termination when more than one solution is found.
- Discard puzzles that are unsolvable or have multiple solutions.

1.4 Module 4: Difficulty Evaluation and Optimisation

- For each remaining puzzle, compute a difficulty score $\text{Diff}(P)$ leveraging:
 - branching factor / search tree statistics,
 - logical deduction counts,
 - candidate entropy measures such as

$$\sum_{i=1}^{81} \log(|C_i(P)|).$$

- Identify the grid G^* that yields the maximal difficulty:

$$G^* = \arg \max_G \text{Diff}(P(m, G)).$$

- Optionally refine the search using local improvement heuristics such as digit permutations or simulated annealing.

1.5 Module 5: Validation and Output

- Re-validate unique solvability using an ILP-based Sudoku solver.
- Produce a final difficulty analysis, including:
 - search tree depth,
 - backtracking counts,
 - logical reasoning metrics.
- Output the final puzzle $P(m, G^*)$, its solution, and its difficulty score.

1.6 Summary Pipeline

Generate Full Grids $\rightarrow$ Apply Pattern $m \rightarrow$ Check Uniqueness $\rightarrow$ Compute Difficulty $\rightarrow$ Select Best G^*
